

Invited User Registration — Current Mobile Logic (Backend Handover)

Oct 2, 2026 · @ahmed nasr

Purpose

An invited user registers with an 8-character invitation code that an owner (or PHD) created. The code decides the role, and the phone runs all the logic: it reads the invitation, picks the role, creates the user, closes the invitation, and uploads documents. **There is no OTP in this flow:** the mobile number comes from the invitation and is never verified.

How every call travels today. Same as owner registration: the app encrypts `{ Endpoint, Method, Body }` and POSTs it to `CommunityAppProxy/HandleRequest`, which forwards the raw OData call to CRM (`.../api/data/v9.1/`) with no business rules.

Scope. Join Palm Hills → Invited user → enter code, for every role the code can carry:

com_to on the invitation	Role	Screens used	com_registered as set
1	Family member	Family member details → Check identity	2
21	Tenant family member	Family member details → Check identity	5
2	Tenant	Tenant page (details tab → documents tab)	3
4	Ref	Tenant page	6
41	Ref owner	Tenant page	7
3	Household	Disabled (code shows "Wrong Invitation Code")	—

Also covered: "Request an invitation from the owner" (no code yet), and resuming from login.

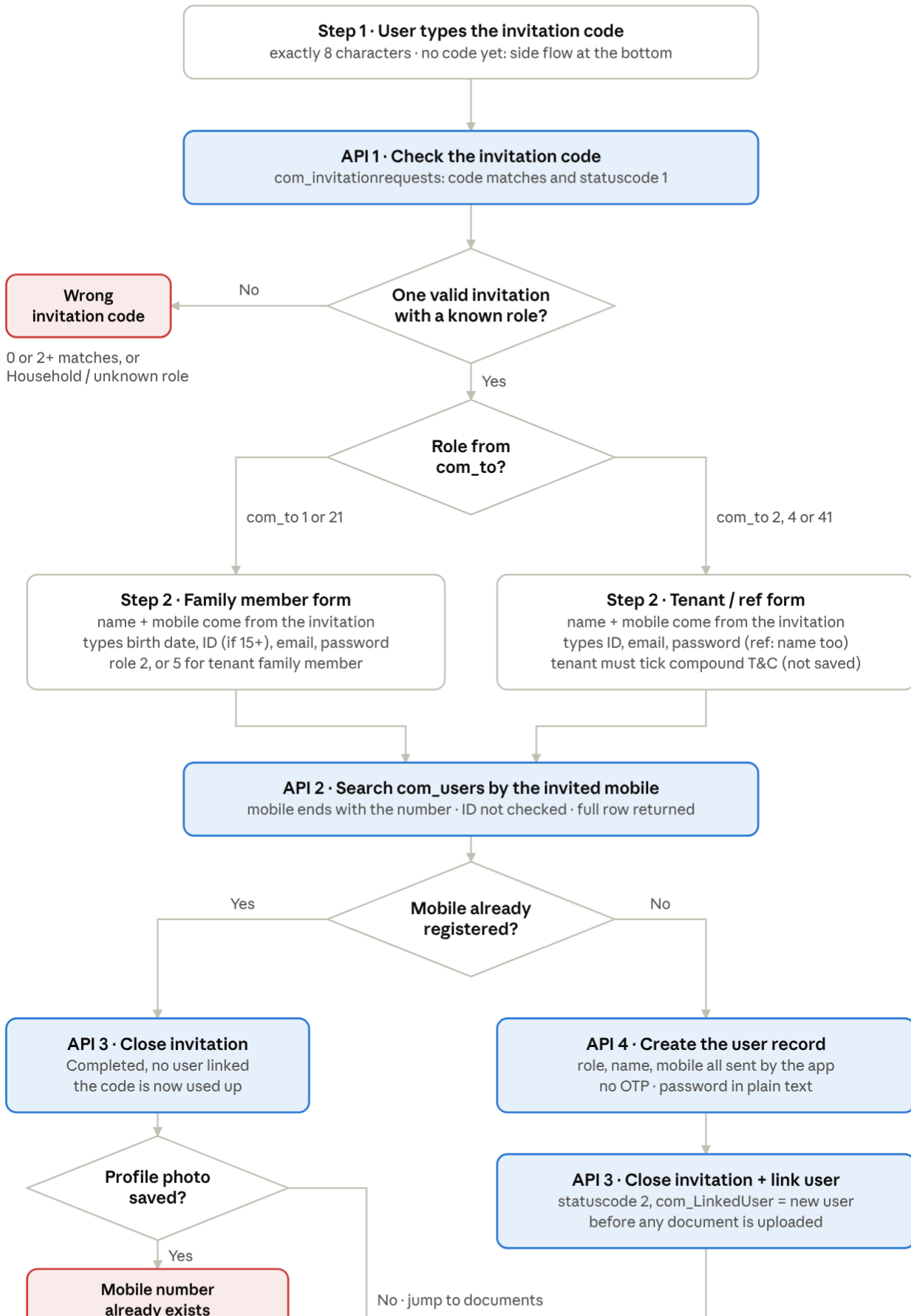
Source: `lib/screens/authentication_cycle/join by code/`, `.../family member/`, `.../tenant/`, `.../take selfy/`, `lib/repository/registration_repository.dart`, `lib/screens/login/controller/login_controller.dart`.

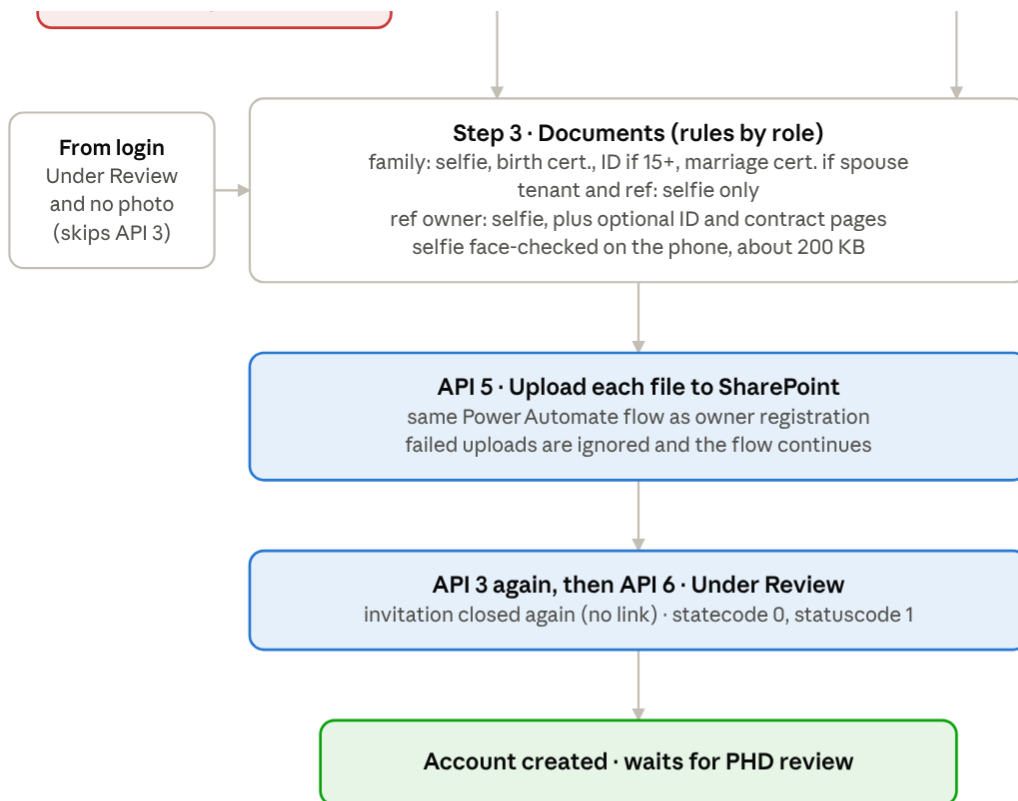
Current flow

Invited user: the code alone decides who you are, with no OTP

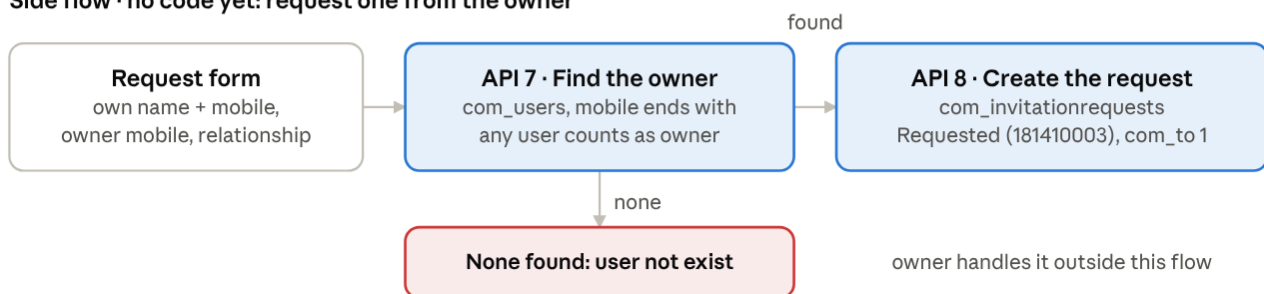
Numbers match the API sections below. Every call goes through the CommunityApp proxy.

○ App screen or app-only logic ○ API call through the proxy ○ Flow stops ○ Finished





Side flow · no code yet: request one from the owner



invited-user registration as the app runs it today · 2 role paths, 4 decisions, 8 API calls

The code picks the role and the invitation supplies the name and mobile; nothing proves the user owns that mobile. The invitation is closed before any document is uploaded.

Step by step: each API call and its business rule

The app makes up to 8 different calls. Numbers match the chart above. Family-member and tenant paths use the same calls with different rules; differences are marked.

Step 1 · User types the invitation code (app only)

Exactly 8 characters; the Continue button stays disabled until then.

API 1 · Check the invitation code

Call	GET com_invitationrequests
Query	<code>\$select=com_familymembermobilenumber,com_familymembername,com_relationship,com_to &\$expand=com_com_invitationrequest_com_invitationunit_InvitationRequest(\$select=com_invitationunitid;\$expand=com_Unit(\$select=com_unitid;\$expand=com_Compound(\$select=com_tenancytermsandconditionsfortenants))) &\$filter=(com_code eq '{code}' and statuscode eq 1)</code>
Response read	com_invitationrequestid, com_familymembername, com_familymembermobilenumber, com_relationship, com_to, and each unit's compound com_tenancytermsandconditionsfortenants
Rule	Valid only if exactly one invitation has this code and <code>statuscode = 1</code> (Confirmed). 0 or 2+ matches → "Wrong Invitation Code". Then <code>com_to</code> picks the role (table in Purpose); an unknown <code>com_to</code> (incl. 3 Household) → "Wrong Invitation Code". Compound T&C texts are collected for the tenant screen.

Step 2 · Details form (app only)

Name and mobile are **taken from the invitation**, not typed (ref and ref owner can retype the name). The user types:

- **Family member / tenant family member:** birth date; national ID or passport (hidden when the birth date makes them under 15); email; password.
- **Tenant / ref / ref owner:** national ID or passport, email, password (+ name for ref and ref owner).
- **Tenant only:** must tick "agree to Terms and Conditions" when the compound has T&C text. The acceptance is **not saved** anywhere.
- Same field rules as owner: national ID exactly 14 digits, email regex, password needs digit + lower + upper + symbol.

API 2 · Search existing app users by the invitation's mobile

Call	GET <code>com_users</code>
Query	<code>\$filter=endswith(com_mobilenumber, '{mobile}')</code> — <code>{mobile}</code> is the invitation's number with its first 2 characters (<code>+2</code>) cut off
Response read	first record's <code>com_userid</code> , <code>com_profilepicturefilepath</code> (no <code>\$select</code> : the full row, incl. <code>com_password</code> , comes back)
Rule	Mobile only. National ID / passport are not checked for duplicates.

Decision on the result:

- **No record** → API 4 (create).
- **Record found, photo path filled** → API 3 closes the invitation **without** linking the user, then "Mobile number already exist". The code is now used up.
- **Record found, photo path empty** → API 3 closes the invitation **without** linking the user, then the app jumps to the documents step for that existing record ("User exists but with incomplete profile").

API 3 · Close the invitation

Call	PATCH <code>com_invitationrequests({invitationId})</code>
Body	<code>statecode: 1</code> , <code>statuscode: 2</code> (Completed), <code>com_acceptedon</code> = now (UTC), and <code>com_LinkedUser@odata.bind:</code> <code>"com_users({userId})"</code> only after a new user is created
Used when	(a) right after API 4 — with the link; (b) the existing-user cases above — without the link; (c) again on final submit — without the link
Rule	The invitation is marked Completed before documents are uploaded. If the user quits now, the code can't be reused.

API 4 · Create the user record

Call	POST <code>com_users</code>
Body	<code>com_name</code> (from invitation, or typed for ref / ref owner), <code>com_mobilenumber</code> (from invitation), <code>com_nationalid</code> or <code>com_passportnumber</code> , <code>com_email</code> , <code>com_password</code> (plain text), <code>com_birthdate</code> (family member only), <code>statuscode: 1</code> , <code>com_appnotificationtoken</code> , <code>com_otpexpirydate</code> = now + 120 s (unused), <code>com_registered</code> from the role table
Response read	<code>com_userid</code> (and the full record)
Rule	No <code>com_requestotp</code> : no SMS, no verification. Tenant path falls back to <code>com_registered</code> : 2 if the role is missing.

Step 3 · Documents (app only)

Selfie is taken in-app, face-checked on the phone (same ML Kit rules as owner) and compressed to ~200 KB. Other files are picked from the gallery as base64.

Role	Required	Optional
Family member / tenant family member	selfie; birth certificate; ID front if age ≥ 15; marriage certificate if <code>com_relationship</code> = 181410000 (Spouse)	—
Tenant, ref	selfie	—
Ref owner	selfie	ID front, contract page 1, contract page 2

Age is computed on the phone from the typed birth date ($\text{days} / 365 < 15$).

API 5 · Upload each file to SharePoint

Same Power Automate flow and body as owner registration (POST `{sharepointattachmentcreationendpointurl}`, `attachmentid` = `userid`, `entityname`: `"com_users"`, `EnableAuthorization`: `false`).

File	CRM name / path fields
Selfie	<code>com_profilepicturefilename</code> / <code>com_profilepicturefilepath</code>
ID or passport front	<code>com_nationalidfrontname</code> / <code>com_nationalidfrontpath</code>
Birth certificate	<code>com_birthcertificatefilename</code> / <code>com_birthcertificatefilepath</code> (file sent as marriage.png by mistake)
Marriage certificate	<code>com_marriagecertificatename</code> / <code>com_marriagecertificatepath</code>
Contract page 1 / 2 (ref owner)	<code>com_contractfrontname</code> / <code>com_contractfrontpath</code> , <code>com_contractbackname</code> / <code>com_contractbackpath</code>

Family path: all files in parallel, results ignored. Tenant path: one by one, a failure is logged and skipped. Either way the flow continues to API 6.

API 6 · Set status to Under Review

Call	<code>PATCH com_users({userId})</code>
Body	<code>{ "statecode": 0, "statuscode": 1 }</code>
Rule	Runs after uploads (and after API 3 again), then "Account created". Already <code>statuscode</code> 1 from creation.

Side flow · Request an invitation from the owner

For a family member without a code. The user types their name, their mobile, the owner's mobile and the relationship (Spouse, Child, Parent, Sibling).

	API 7 · Find the owner	API 8 · Create the invitation request
Call	<code>GET com_users</code>	<code>POST com_invitationrequests</code>

	API 7 · Find the owner	API 8 · Create the invitation request
Query / body	<code>\$filter=(endswith(com_mobilenumbers, '{ownerMobile}'))</code>	<code>com_familymembername, com_familymembermobilenumbers = + {dial}{mobile}, com_relationship, com_InvitationProvider@odata.bind: "com_users({ownerId})", statuscode: 181410003 (Requested), com_to: 1</code>
Rule	Any user with that mobile counts as "the owner" (role not checked). None → "this user not exist"	How the owner approves it and the code is issued is outside this flow (not covered here). A 60 s timer on the phone is the only spam limit

Side entry · Resume from login

Login (same query as owner: mobile + `com_password eq`) finds `statuscode 1` and no profile photo, and offers to finish. By `com_registered as : 2 or 5` → family Check identity page; 3, 6, 7 → tenant page on the documents tab. No invitation id is known here, so API 3 is skipped.

Logic the backend should own

The biggest gap: **whoever holds the 8-character code becomes that person**, with no proof they own the invited mobile number. Everything else below is decided on the phone today.

#	Decided on the phone today	Problem	Move to the API
1	Code is valid (API 1)	The code is the only proof of identity; no OTP to the invited mobile; no limit on guesses	Server validates the code with rate limits, then sends an OTP to the invitation's mobile
2	Role from <code>com_to</code>	App maps <code>com_to</code> → role and sends <code>com_registered as</code> itself; a client can register as any role	Server derives the role from the invitation
3	Name and mobile from the invitation	App sends them back in API 4; a client can change them	Server copies them from the invitation

#	Decided on the phone today	Problem	Move to the API
4	User exists (API 2)	Mobile only, <code>endsWith</code> after cutting 2 characters; ID not checked; full row incl. <code>com_password</code> returned	Exact E.164 match on mobile, plus ID duplicate check, <code>\$select</code> only
5	Invitation closed early (API 3)	Marked Completed before documents; existing-user cases burn the code without linking anyone	Close and link only on successful submit
6	Existing unfinished user	App jumps to documents for that record with no check that it belongs to this invitation	Server checks the record matches the invitation (mobile, role)
7	Tenant T&C accepted	Checked on the phone, never saved	Server stores acceptance (who, when, which compound text)
8	Under-15 rule	Age from the typed birth date decides if ID is needed	Server computes age and the required documents
9	Required documents per role	Only on the phone; uploads unchecked; birth certificate sent as <code>marriage.png</code>	Server validates per role and checks each upload
10	Uploads (API 5)	Any <code>attachmentid</code> , <code>EnableAuthorization: false</code>	Server-side uploads for the token's user only
11	Under Review (API 6)	Already set at creation; set even after failed uploads	Server sets it once all checks pass
12	Request invitation (API 7, 8)	Any user with that mobile counts as "owner"; full rows returned; spam limited only by a 60 s phone timer	Server checks the target is an owner, rate-limits, never returns user rows
13	Login resume	Password compared on the phone; resume step chosen on the phone	Server returns <code>nextStep</code>

Backend build guide

The goal: the app sends the code, what the user typed and the files; the API reads the invitation and decides role, rules and next step. Endpoint names are a proposal and share the OTP and documents endpoints with the owner registration guide.

Rules for whoever builds it

- 1. The invitation is the source of truth.** Role (`com_to` → `com_registeredas`), name, mobile, relationship and units come from the invitation record, never from the request.
- 2. Prove the mobile.** Before creating or reusing a user, send an OTP to the invitation's mobile and verify it server-side (new step; see Open questions).
- 3. Never return a secret or another user's row.** `$select` on every read; no `com_password`, `com_otp`, invitation code or full user rows in any response. Mask the invited mobile (`+20 10●●●●45 67`).
- 4. Escape every OData value** and validate format first (code: exactly 8 characters).
- 5. Close the invitation only on success.** `statuscode: 2`, `statecode: 1`, `com_acceptedon`, `com_LinkedUser` are set in the same step that moves the user to Under Review.
- 6. Required documents are computed by the server** from role, relationship and age.
- 7. Registration token, not record ids.** The token is bound to one invitation and, later, one `com_userid`.
- 8. Keep CRM as is.** Same entities, fields, option-set values and SharePoint flow.

Proposed endpoints

Endpoint	Replaces	Does
POST <code>/registration/invitation/check</code>	API 1 + role switch	Validates the code (rate-limited). Returns <code>role</code> , <code>name</code> , <code>maskedMobile</code> , <code>relationship</code> , <code>termsAndConditions[]</code> , the form fields to show, and <code>registrationToken</code>

Endpoint	Replaces	Does
POST <code>/registration/invitation/start</code>	API 2, 4 (+ first OTP)	Takes ID type/number, email, password, birth date (family), name (ref/ref owner), <code>termsAccepted</code> (tenant). Checks existing user, creates or reuses the record, sends the OTP. Returns <code>nextStep: "verify-otp"</code>
POST <code>/registration/otp/resend</code> , <code>/otp/verify</code>	New	Same as owner guide. On success <code>nextStep: "upload-documents"</code> + <code>requiredDocuments[]</code>
POST <code>/registration/documents</code> (multipart)	API 5, 3, 6	Validates documents per role, uploads, links and closes the invitation, sets Under Review
POST <code>/registration/invitation/request</code>	API 7 + 8	Side flow: checks the target mobile belongs to an owner, creates the request (Requested), rate-limited
Login response <code>nextStep</code>	Side entry	Same as owner guide

Per query: why it moves and what the backend must do

API 1 · Check the invitation code

Why it moves

- The code is the only proof of identity, and the app can try codes without limit.
- The response hands the phone the invited person's full mobile and name before anything is proven.
- The app maps `com_to` to a role; an unknown value or Household is silently "wrong code".

Backend must (`/invitation/check`)

- Validate format (8 characters), escape, query with the same `$select` / `$expand` and `statuscode eq 1`. Exactly one match, else `INVITATION_INVALID`.
- Map `com_to` → role server-side: 1 → 2, 21 → 5, 2 → 3, 4 → 6, 41 → 7; 3 (Household) → `INVITATION_ROLE_NOT_SUPPORTED`.

- Rate-limit by device/IP and by code (e.g. 5 failures → lock for 15 min).
- Return the masked mobile, name, role, relationship, T&C texts, form fields, token.

Done when: guessing codes is throttled; the full mobile never reaches the phone.

API 2 · Search existing users

Why it moves

- `endswith` on the number with its first 2 characters cut assumes `+2X` ; for other countries it matches the wrong length.
- National ID / passport duplicates are never checked.
- No `$select` : the full user row, with `com_password` , reaches the phone.

Backend must (inside `/invitation/start`)

- Exact match on the invitation's mobile (E.164) **or** the typed ID;
`$select=com_userid,com_profilepicturefilepath,com_mobileverified,com_registered,com_statuscode` .
- Complete user → `USER_EXISTS` and leave the invitation open (see Open questions).
- Unfinished user → reuse it only if its mobile equals the invitation's mobile and its role matches.

Done when: no user row leaves the server; an ID already registered returns `USER_EXISTS` .

API 3 · Close the invitation

Why it moves

- Closed (Completed) before documents; a user who quits can't use the code again.
- In both existing-user cases it is closed without `com_LinkedUser` , so CRM loses who used it.
- Any client can PATCH any invitation id.

Backend must

- Close it only inside `/registration/documents` , after Under Review is set, with `com_LinkedUser` , `statecode: 1` , `statuscode: 2` , `com_acceptedon` (UTC).

Done when: every Completed invitation has a linked user; quitting before submit leaves the code usable.

API 4 · Create the user record

Why it moves

- Role, name, mobile and status are all sent by the app.

- Password in plain text; no OTP; `com_otpexpirydate` set for nothing.
- Tenant path defaults to role 2 (family member) if the role is missing.

Backend must

- Create with name and mobile from the invitation (ref / ref owner may override the name), role from `com_to`, `com_birthdate` for family roles, hashed password, `com_requestotp: true` (new), `$select=com_userid` only.
- For tenants, store T&C acceptance (field to confirm).

Done when: a request can't change role, mobile or status.

Step 3 + API 5 · Upload documents

Why it moves

- Required-document rules (role, spouse, age 15) live only on the phone; age comes from a typed date.
- Uploads to any `attachmentid` with authorization off; results ignored (family) or skipped (tenant).
- Birth certificate is uploaded with the name `marriage.png`.

Backend must (`/registration/documents`)

- Require a verified mobile. Compute required documents: family roles → selfie + birth certificate + ID front if age ≥ 15 (from `com_birthdate`) + marriage certificate if `com_relationship` = 181410000; tenant / ref → selfie; ref owner → selfie (+ optional ID front, contract pages).
- Upload server-side with correct file names; any failure → `UPLOAD_FAILED`, no status change.

Done when: a missing required document returns `DOCUMENTS_MISSING` with the list; a failed upload never reaches Under Review.

API 6 · Set Under Review

Why it moves: set by the app, already set at creation, and set even after failed uploads.

Backend must: set `statecode: 0`, `statuscode: 1` at the end of `/registration/documents`, then close the invitation (API 3) in the same step.

Done when: Under Review implies verified mobile + required documents + Completed invitation.

API 7 + 8 · Request an invitation from the owner

Why it moves

- Any user with that mobile is treated as the owner (could be a tenant or another family member).
- The full user row comes back to the phone.
- Only a 60 s phone timer limits requests; anyone can flood an owner.

Backend must (/invitation/request)

- Find the target by exact mobile with `com_registeredas = 1` (Owner) and an active status; `$select=com_userid` only. None → `OWNER_NOT_FOUND`.
- Create `com_invitationrequests` with `statuscode: 181410003`, `com_to: 1`, the provider link, name, E.164 mobile, relationship (181410000–181410003 only).
- Rate-limit per requester mobile and per owner; reject a duplicate open request.

Done when: a non-owner mobile returns `OWNER_NOT_FOUND` ; repeat requests are throttled server-side.

Side entry · Resume from login

Why it moves: password compared on the phone; resume screen picked from `com_registeredas` on the phone; no OTP.

Backend must: return `nextStep` and a token (`verify-otp` if not verified, else `upload-documents`), plus `requiredDocuments[]`.

Done when: the app needs no role logic to resume.

Reference

`com_invitationrequests` fields

Field	Meaning
<code>com_invitationrequestid</code>	Record id
<code>com_code</code>	The 8-character invitation code
<code>com_to</code>	Invited as: 1 Family member, 21 Tenant family member, 2 Tenant, 3 Household, 4 Ref, 41 Ref owner
<code>com_familymembername</code> , <code>com_familymembermobilenumber</code>	Invitee's name and mobile (<code>+{dial}{mobile}</code>)

Field	Meaning
<code>com_relationship</code>	181410000 Spouse, 181410001 Child, 181410002 Parent, 181410003 Sibling, 181410004 Tenant
<code>com_InvitationProvider</code>	Lookup to the inviting <code>com_users</code> record
<code>com_LinkedUser</code>	Lookup to the registered <code>com_users</code> record
<code>com_acceptedon</code>	When the invitation was used
<code>com_com_invitationrequest_com_invitationunit_InvitationRequest</code>	Invitation units → <code>com_Unit</code> → <code>com_Compound.com_tenancytermsandconditionsfortenants</code>

Invitation status values

statuscode	Meaning
181410003	Requested (created by the side flow)
1	Confirmed — the only status API 1 accepts
181410005	Under review
2	Completed (with <code>statecode</code> 1) — set by API 3
181410002	Rejected by owner
181410006	Rejected by user
181410004	Rejected by community team
181410000	Deactivated

`com_users` fields added by this flow

Same fields as the owner guide, plus: `com_birthdate` (family roles), `com_birthcertificatefilename` / `...filepath`, `com_marriagecertificatename` / `...path`, `com_contractfrontname` / `...frontpath`, `com_contractbackname` / `...backpath`. `com_registeredas`: 2 Family member, 3 Tenant, 5 Tenant family member, 6 Ref, 7 Ref owner. User status values are the same as the owner guide (1 Under Review, 181410000 Approved, 181410001 Rejected, 2 Deactivated).

Error codes and today's app messages

Code	HTTP	App shows today
VALIDATION_FAILED	400	Field errors
INVITATION_INVALID	404	"Wrong Invitation Code"
INVITATION_ROLE_NOT_SUPPORTED	422	"Wrong Invitation Code" (Household)
INVITATION_RATE_LIMITED	429	New
TERMS_NOT_ACCEPTED	400	"You must agree to Terms And Conditions"
USER_EXISTS	409	"Mobile number already exist"
INCOMPLETE_PROFILE	200 (info)	"User exists but with incomplete profile"
OTP_INVALID / OTP_EXPIRED / OTP_ATTEMPTS_EXCEEDED	400 / 400 / 429	New (no OTP today)
DOCUMENTS_MISSING	400	"You Must Take Selfie" / "... National ID" / "... Birth Certification" / "... Marriage Certificate"
UPLOAD_FAILED	502	None today (failures are ignored)
OWNER_NOT_FOUND	404	"this user not exist"
INVITATION_REQUEST_FAILED	502	"error send invitation"
REGISTRATION_TOKEN_INVALID	401	New

Open questions to confirm before building

- ☐ Add an OTP to the invitation's mobile? (Recommended; today there is none, so the code alone proves identity.)
- ☐ When the invited mobile already has a complete account: keep the invitation open, link it to that user, or close it? (Today: closed with no link.)
- ☐ Where to store tenant T&C acceptance (field on `com_users` or on the invitation)?
- ☐ Should ref / ref owner still be allowed to change the name from the invitation?
- ☐ Is Household (`com_to` 3) dropped for good?

- ☐ Who may receive a "request invitation": owners only, or tenants too (for tenant family members)?
- ☐ Code guessing limits: attempts per device/IP and lock time.
- ☐ Ref owner: are ID front and contract pages really optional?